

CollabCoder: A GPT-Powered WorkFlow for Collaborative Qualitative Analysis

Jie Gao

Singapore University of Technology and Design
Singapore
gaojie056@gmail.com

Toby Jia-Jun Li

University of Notre Dame
Notre Dame, IN, USA
toby.j.li@nd.edu

Yuchen Guo

Singapore University of Technology and Design
Singapore
yuchen_guo@mymail.sutd.edu.sg

Simon Tangi Perrault

Singapore University of Technology and Design
Singapore
perrault.simon@gmail.com

ABSTRACT

Collaborative Qualitative Analysis (CQA) process can be time-consuming and resource-intensive, requiring multiple discussions among team members to refine codes and ideas before reaching a consensus. We introduce CollabCoder, a system leveraging Large Language Models (LLMs) to support three CQA stages: independent open coding, iterative discussions, and the development of a final codebook. In the independent open coding phase, CollabCoder provides AI-generated code suggestions on demand and allows users to record coding decision-making information (e.g. keywords and certainty) as support for the process. During the discussion phase, CollabCoder helps to build mutual understanding and productive discussion by sharing coding decision-making information within the team. It also helps to quickly identify agreements and disagreements through quantitative metrics, in order to build a final consensus. During the code grouping phase, CollabCoder employs a top-down approach for primary code group recommendations, reducing the cognitive burden of generating the final codebook. The **source code** for CollabCoder can be accessed via GitHub at <https://github.com/gaojie058/CollabCoder>.

CCS CONCEPTS

• **Human-centered computing** → **Collaborative and social computing systems and tools**.

KEYWORDS

collaborative qualitative analysis, GPT, mutual understanding, qualitative coding

ACM Reference Format:

Jie Gao, Yuchen Guo, Toby Jia-Jun Li, and Simon Tangi Perrault. 2023. CollabCoder: A GPT-Powered WorkFlow for Collaborative Qualitative Analysis. In *Computer Supported Cooperative Work and Social Computing (CSCW '23 Companion)*, October 14–18, 2023, Minneapolis, MN, USA. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3584931.3607500>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CSCW '23 Companion, October 14–18, 2023, Minneapolis, MN, USA

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0129-0/23/10...\$15.00

<https://doi.org/10.1145/3584931.3607500>

Companion), October 14–18, 2023, Minneapolis, MN, USA. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3584931.3607500>

1 INTRODUCTION

While Collaborative Qualitative Analysis (CQA) is critical to ensuring the credibility, rigor, and representativeness of the interpretation, research has found it time-consuming and effort-demanding due to its iterative nature [1, 4, 10, 15].

In the past few years, various prototypes and systems have been proposed to explore the potential of applying artificial intelligence (AI) in individual qualitative analysis to assist in the process of code proposition [3, 5, 7–9, 13]. However, AI's application in CQA is still lacking, as the majority of current CQA assistance continues to depend on traditional methods [2, 4, 15].

As Large Language Models (LLMs) such as OpenAI's GPT series¹ advance and demonstrate exceptional capabilities in understanding and generating text [11, 14, 16], we introduce CollabCoder, a prototype incorporating the GPT-3.5 model into key CQA stages [5, 12], which:

- (1) Offers AI-generated code suggestions and allows users to document their coding decision-making information (e.g., keywords and certainty) during the **individual open coding phase**, aiding in formulating the preliminary open codes;
- (2) Shares decision-making data and provides quantitative metrics to swiftly identify (dis)agreements during the **code merging and discussion phase**, facilitating the formulation of final code decisions;
- (3) Implements a top-down approach to organizing codes during the **code group generation phase**, leading to the establishment of final code groups.

2 COLLABCODER SYSTEM

We introduce an example scenario to demonstrate the usage of CollabCoder. Suppose two coders **Alice** and **Bob** are conducting qualitative coding for their data. The lead coder, **Alice**, first creates a new project on CollabCoder, then imports the data, specifies the level of coding as "paragraph", and invites **Bob** to join the project. After clicking on CREATE PROJECT, CollabCoder's parser will split the imported raw data into units (paragraph in this case). The project can then be shown on both coders' interfaces.

¹<https://platform.openai.com/docs/models/gpt-3>

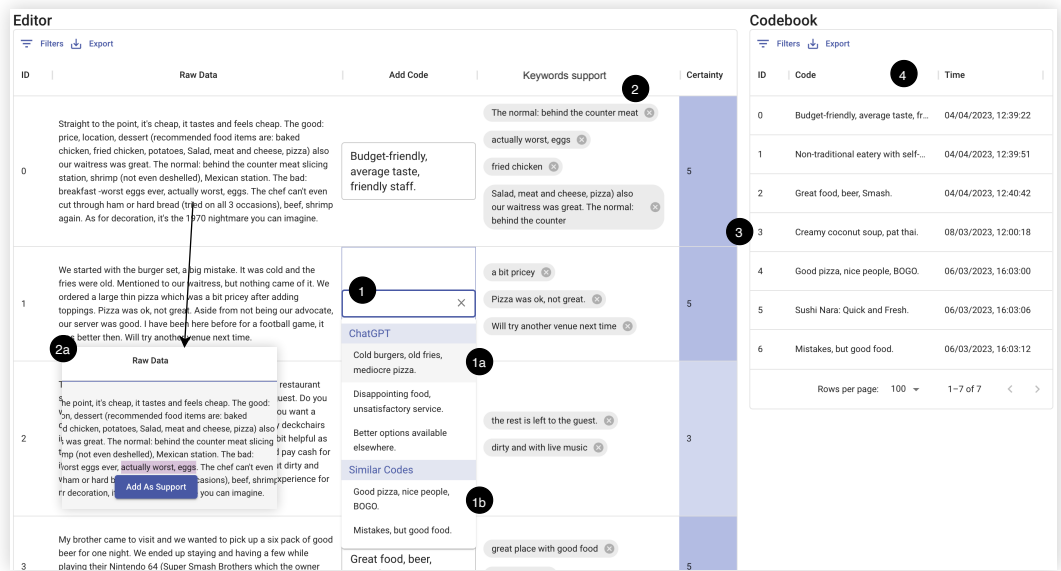


Figure 1: Phase 1–Independent Open Coding. The interface enables users to assign codes to data units: 1) input customized codes, 1a) choose from the GPT's recommendations, 1b) choose from the top three relevant codes extracted from code history; 2) record supporting keywords or phrases by 2a) selecting from raw data and clicking on "Add As Support", relevant keywords will be displayed in the adjacent column to the code; 3) assign a certainty level ranging from 1 to 5, where 1="not certain", 5="very certain"; and 4) review and modify the individual codebook.

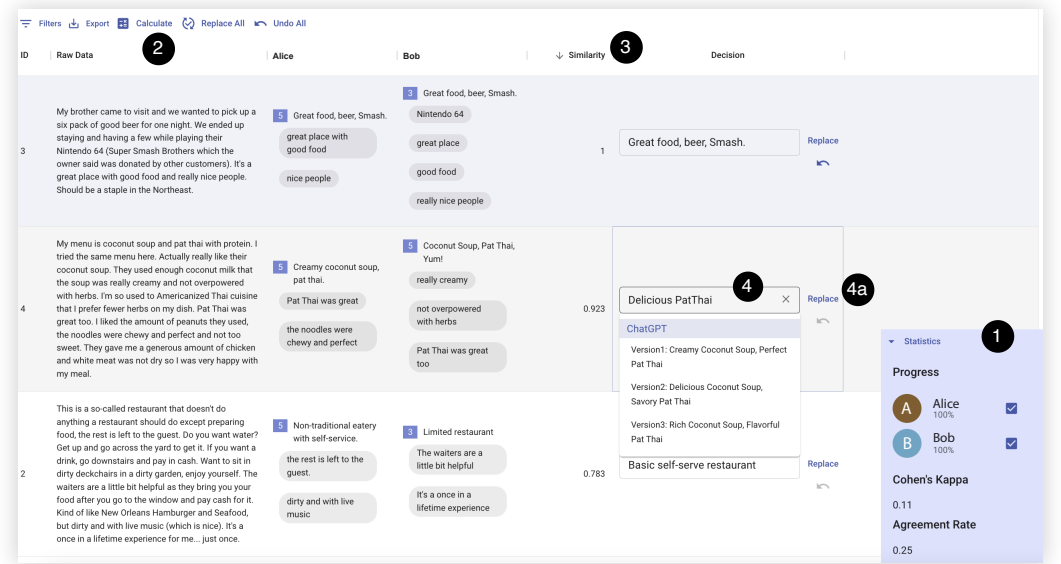


Figure 2: Phase 2–Code Merging and Discussion. Users can compare and reach a consensus in Phase 2: 1) reviewing another coder's progress and clicking on the checkbox once both individuals complete their coding, 2) computing the similarity between code pairs, 3) sorting the similarity scores from highest to lowest and identifying agreements, and 4) making a decision based on both initial codes and raw data through discussion or selecting and modifying GPT's code decision suggestions. Additionally, users have the option to 4a) substitute the original codes proposed by two coders and revert back to the original codes if required. They can also replace or revert all code decisions with a single click on the top bar.

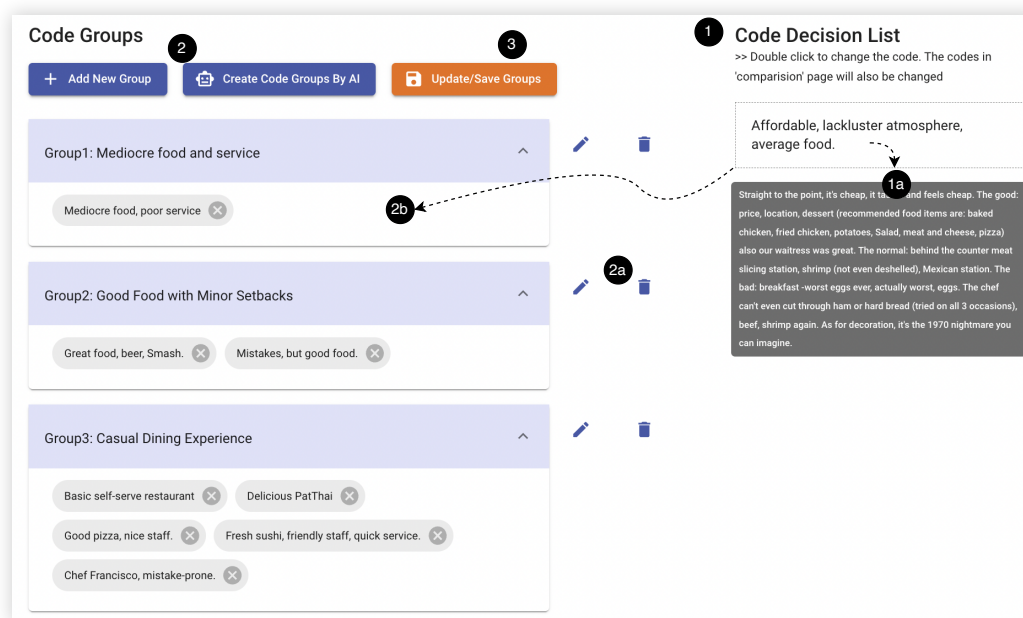


Figure 3: Phase 3—Code Group Generation. The interface enables users to manage their code decisions in Phase 3: 1) code decisions in Phase 2 are automatically compiled into a list of unique codes that users can edit by double-clicking and accessing the original data by hovering over the code. 2) Users can group their code decisions by using either "Add New Group" or "Create Code Groups By AI" options. They can then 2a) modify the code group name, and 2b) drag the code decisions into code groups. 3) Finally, users can save and update the code groups.

2.1 Phase 1: Independent Open Coding

In the independent open coding phase, **Alice** and **Bob** individually formulate codes for each unit in their separate workspaces via the same interface (Figure 1). Their work is done independently, with no visibility into each other's codes. ① **Proposing codes:** If **Alice** wants to propose a code for a sentence describing food, she can either *craft* her own code (e.g., "Quiet but bad"), choose from code recommendations provided by the GPT model (e.g., "Quiet but bad service", "Wrong choice of food", "Negative eating experience"), or *pick* one of the top three most relevant codes discovered in her code history (e.g., "Mistake and bad experience"), and making modifications as needed. ② **Recording decision-making information:** She can *select* relevant keywords/phrases (e.g., "made mistakes", "the restaurant was quiet") from the RAW DATA cell that supports her proposed code, which will be added into the KEYWORDS SUPPORT cell beside her proposed code. She can also *assign* a CERTAINTY, ranging from 1 to 5, to the code. This newly generated code will be included in **Alice's** personal CODEBOOK and can be *viewed* at any time. ③ **Understanding team progress:** They can *check* the progress of each other in the PROGRESS at any time (see Figure 2).

2.2 Phase 2: Code Merging and Discussion

Figure 2 depicts the shared workspace where coding teams collaborate, discussing their code choices and making final decisions regarding the codes in Phase 2. ① **Coding independence:** After

completing her coding, **Alice** can *check* the checkbox next to **Bob's** name only if she sees that his progress is at 100%. ② **Filtering (dis)agreements:** Subsequently, she can *click* the CALCULATE button to generate quantitative metrics such as similarity scores and IRR (Cohen's Kappa and Agreement Rate) within the team. The rows can then be sorted according to the ranking of the similarity scores from the highest to the lowest. ③ **Merging codes by discussion:** **Alice** can share her screen via a Zoom meeting with **Bob** to *compare and discuss* their codes, starting from code pairs with high similarity scores. For instance, **Alice's** code "Quite but bad" with a certainty of 4 includes "waitress made mistakes" and "the restaurant was quiet" supports, while **Bob's** code "Quiet but lazy service" with a certainty of 5 includes "the restaurant was quiet" and "service was lazy" as KEYWORDS SUPPORT. The similarity score between their codes could be 0.9, showing a high agreement. During the discussion, they both agreed that the final code should contain the word "service" due to their similar KEYWORDS SUPPORT. Eventually, they agree on the merged code "quiet but bad service", but if they cannot reach a consensus about the final expression of the code, they can ask GPT to provide suggestions (e.g., "quiet but wrong service"). However, if the code pair presents a low similarity score, they must allocate additional time to scrutinize the code decision information and identify the keywords that led to differing interpretations. ④ **Update IRR metrics:** Once all code decisions have been made, **Alice** can then *click* on REPLACE to replace the original codes, resulting in an

update of Cohen's Kappa and Agreement Rate. This action can be undone by clicking on UNDO.

2.3 Phase 3: Code Group Generation

Once **Alice** and **Bob** have agreed on the final code decisions for all the units, the code decision list will be displayed in the code grouping phase, as shown in Figure 3. This interface, along with its corresponding data (i.e., decision list and code groups), is shared uniformly among the coding team. ① **Code decision list:** For further discussion, **Alice** can continue to share her screen with **Bob** on Zoom. She can *hover over* each CODE DECISION to refer to the corresponding raw data and *double-click* to edit the code decision. ② **Generating code group:** **Alice** and **Bob** can collaborate to propose the final code groups by *clicking* on ADD NEW GROUP and *drag* the code decisions into code groups. For instance, a group name like "Good food with minor setbacks" can include "Good food, beer, Smash" and "Mistakes but good food". Alternatively, they can request GPT assistance by *clicking* on the CREATE CODE GROUPS BY AI to automatically generate several code groups and place the individual code decisions into them. These groups can still be manually adjusted by coders. Once they finish grouping, they can proceed to report their findings as necessary.

3 LIMITATIONS AND FUTURE WORK

There are several aspects where CollabCoder could be improved in future work: ① **Data segmentation:** CollabCoder employed only pre-defined unit data to ensure coders agree on code units, facilitating comparison and computation of IRR metrics within the coding team. Nevertheless, it was not designed for complex, time-intensive data (e.g., interview data) that requires human-led segmentation into units. Future work could investigate the use of GPT for segmenting interview data into semantic units and automating the data import process. ② **Thoughtful design to prevent over-reliance risk:** Directly showing coders the code suggestions might encourage an over-reliance on the system [5, 6], which could potentially supplant human autonomy in coding. Thus, it could be advantageous to have GPT generate coding insights based on users' coding patterns rather than providing direct suggestions. Moreover, displaying code suggestions after a brief delay could allow coders sufficient time to make their own code decisions, in order to keep their autonomy. ③ **Addressing multiple codes for a single data unit:** Tackling the occurrence of multiple codes for one unit is vital for achieving a comprehensive interpretation, especially among the coding team. This aspect must be given careful consideration when designing a qualitative analysis tool, especially when collaboration is involved.

4 CONCLUSION

In conclusion, CQA is crucial but time-consuming. We propose CollabCoder, a prototype using the GPT-3.5 model, to streamline the CQA process. CollabCoder can provide AI-generated code suggestions, document decision-making information, filter areas of agreement and disagreement, aid in final code decision-making, and simplify the codebook creation process. Our work paves the way for the development of future LLM-empowered CQA tools.

REFERENCES

- [1] Jessica Díaz, Jorge Enrique Pérez-Martínez, Carolina Gallardo, and Ángel González-Prieto. 2021. Applying Inter-rater Reliability and Agreement in Grounded Theory Studies in Software Engineering. *CoRR* abs/2107.11449 (2021). arXiv:2107.11449 <https://arxiv.org/abs/2107.11449>
- [2] Margaret Drouhard, Nan-Chen Chen, Jina Suh, Rafal Kocielnik, Vanessa Peña-Araya, Keting Cen, Xiangyi Zheng, and Cecilia R. Aragon. 2017. Aeonium: Visual analytics to support collaborative qualitative coding. In *2017 IEEE Pacific Visualization Symposium (PacificVis)*. 220–229. <https://doi.org/10.1109/PACIFICVIS.2017.8031598>
- [3] Jessica L. Feuston and Jed R. Brubaker. 2021. Putting Tools in Their Place: The Role of Time and Perspective in Human-AI Collaboration for Qualitative Analysis. *Proc. ACM Hum.-Comput. Interact.* 5, CSCW2, Article 469 (oct 2021), 25 pages. <https://doi.org/10.1145/3479856>
- [4] Abbas Ganji, Mania Orand, and David W McDonald. 2018. Ease on Down the Code: Complex Collaborative Qualitative Coding Simplified with 'Code Wizard'. *Proceedings of the ACM on human-computer interaction* 2, CSCW (2018), 1–24.
- [5] Jie Gao, Kenny Tsu Wei Choo, Junming Cao, Roy Ka-Wei Lee, and Simon Perrault. 2023. CoAlCoder: Examining the Effectiveness of AI-Assisted Human-to-Human Collaboration in Qualitative Analysis. *ACM Trans. Comput.-Hum. Interact.* (aug 2023). <https://doi.org/10.1145/3617362> Just Accepted.
- [6] Jie Gao, Yuchen Guo, Gionnieve Lim, Tianqin Zhang, Zheng Zhang, Toby Jia-Jun Li, and Simon Tangi Perrault. 2023. CollabCoder: A GPT-Powered Workflow for Collaborative Qualitative Analysis. arXiv:2304.07366 [cs.HC]
- [7] Simret Araya Gebreegziabher, Zheng Zhang, Xiaohang Tang, Yihao Meng, Elena L. Glassman, and Toby Jia-Jun Li. 2023. PaTAT: Human-AI Collaborative Qualitative Coding with Explainable Interactive Rule Synthesis. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems (Hamburg, Germany) (CHI '23)*. Association for Computing Machinery, New York, NY, USA, Article 362, 19 pages. <https://doi.org/10.1145/3544548.3581352>
- [8] Matt-Heun Hong, Lauren A. Marsh, Jessica L. Feuston, Janet Ruppert, Jed R. Brubaker, and Danielle Albers Szafrir. 2022. Scholastic: Graphical Human-AI Collaboration for Inductive and Interpretive Text Analysis. In *Proceedings of the 35th Annual ACM Symposium on User Interface Software and Technology (Bend, OR, USA) (UIST '22)*. Association for Computing Machinery, New York, NY, USA, Article 30, 12 pages. <https://doi.org/10.1145/3526113.3545681>
- [9] Jialun Aaron Jiang, Kandrea Wade, Casey Fiesler, and Jed R. Brubaker. 2021. Supporting Serendipity: Opportunities and Challenges for Human-AI Collaboration in Qualitative Analysis. *Proc. ACM Hum.-Comput. Interact.* 5, CSCW1, Article 94 (apr 2021), 23 pages. <https://doi.org/10.1145/3449168>
- [10] Alireza Nili, Mary Tate, Alistair Barros, and David Johnstone. 2020. An approach for selecting and using a method of inter-coder reliability in information management research. *International Journal of Information Management* 54 (2020), 102154. <https://doi.org/10.1016/j.ijinfomgt.2020.102154>
- [11] OpenAI. 2023. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL]
- [12] K Andrew R Richards and Michael A Hemphill. 2018. A practical guide to collaborative qualitative data analysis. *Journal of Teaching in Physical Education* 37, 2 (2018), 225–231. <https://doi.org/10.1123/jtpe.2017-0084>
- [13] Tim Rietz and Alexander Maedche. 2021. Cody: An AI-Based System to Semi-Automate Coding for Qualitative Research. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems (Yokohama, Japan) (CHI '21)*. Association for Computing Machinery, New York, NY, USA, Article 394, 14 pages. <https://doi.org/10.1145/3411764.3445591>
- [14] Ziang Xiao, Xingdi Yuan, Q. Vera Liao, Rania Abdelghani, and Pierre-Yves Oudeyer. 2023. Supporting Qualitative Analysis with Large Language Models: Combining Codebook with GPT-3 for Deductive Coding. In *Companion Proceedings of the 28th International Conference on Intelligent User Interfaces (Sydney, NSW, Australia) (IUI '23 Companion)*. Association for Computing Machinery, New York, NY, USA, 75–78. <https://doi.org/10.1145/3581754.3584136>
- [15] Himanshu Zade, Margaret Drouhard, Bonnie Chinh, Lu Gan, and Cecilia Aragon. 2018. Conceptualizing Disagreement in Qualitative Coding. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems (Montreal QC, Canada) (CHI '18)*. Association for Computing Machinery, New York, NY, USA, 1–11. <https://doi.org/10.1145/3173574.3173733>
- [16] Zheng Zhang, Jie Gao, Ranjodh Singh Dhaliwal, and Toby Jia-Jun Li. 2023. VISAR: A Human-AI Argumentative Writing Assistant with Visual Programming and Rapid Draft Prototyping. arXiv:2304.07810 [cs.HC]